Cours : Les Bases du Langage C#

Par Robert DIASSÉ

1. Les Types de Base en C#

Type	Description	Exemple
`int`	Entier signé sur 32 bits	`int nombre = 42;`
`double`	Nombre à virgule flottante (double précision)	`double pi = 3.14;`
`float`	Nombre à virgule flottante (simple précision)	`float temperature = 25.5f;`
`bool`	Valeur booléenne (true/false)	`bool estVrai = true;`
`char`	Caractère unique	`char initiale = 'A';`
`string`	Chaîne de caractères	`string nom = "Alice";`

Remarque: La déclaration explicite des types est essentielle en C#.

Instructions d'Entrée et de Sortie

Instructions de Sortie

L'instruction principale pour afficher du texte dans la console est `Console.WriteLine()`.

Exemple simple :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Bienvenue en C#!");  // Affiche une ligne
        Console.Write("Affichage sans saut de ligne"); // Reste sur la même ligne
        Console.WriteLine(" - Fin de l'affichage.");
    }
}
```

Explications :

- `Console.WriteLine()` affiche du texte et passe à la ligne suivante.
- `Console.Write()` affiche du texte sans passer à la ligne suivante.

Instructions d'Entrée

Pour lire une entrée utilisateur, on utilise `Console.ReadLine()`.

Exemple : Lecture de texte

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Quel est votre prénom ?");
        string prenom = Console.ReadLine();  // Lecture de l'entrée utilisateur
        Console.WriteLine($"Bonjour {prenom} !");
    }
}
```

Explications :

- `Console.ReadLine()` lit une ligne de texte saisie par l'utilisateur.
- `$"{variable}"` est une interpolation de chaîne permettant d'intégrer directement une variable dans une chaîne de caractères.

Lecture de Types Numériques

Par défaut, `Console.ReadLine()` retourne une chaîne de caractères. Pour lire un nombre, il faut convertir la chaîne.

Exemple : Lecture et Conversion d'un Entier

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Entrez un nombre :");
        int nombre = int.Parse(Console.ReadLine());  // Conversion explicite en
entier
        Console.WriteLine($"Vous avez saisi : {nombre}");
```

```
    }
}
```

Remarque : Gestion des Erreurs

Si l'utilisateur saisit une valeur non valide (par exemple une lettre), une exception sera levée. Une gestion des erreurs est donc recommandée (nous verrons cela dans un cours avancé).

---

2. Les Variables

Une **variable** est un espace mémoire pour stocker une valeur.

Déclaration et initialisation :

```
int age = 25;
double pi = 3.14159;
string message = "Bonjour, monde!";
```

---

3. Les Constantes

Les **constantes** sont des valeurs qui ne changent jamais après leur déclaration.

Déclaration d'une constante :

```
const double Pi = 3.14159;
Console.WriteLine($"La valeur de Pi est {Pi}");
```

---

4. Les Commentaires

C# permet deux types de commentaires :

- **Commentaire sur une ligne :**

```
// Ceci est un commentaire sur une ligne
```

- **Commentaire multi-lignes :**

```
/*
Ceci est un commentaire
sur plusieurs lignes.
*/
```

---

5. Les Opérateurs

Les **opérateurs** permettent de réaliser des opérations sur les variables.

Opérateurs arithmétiques

Opérateur	Description	Exemple
`+`	Addition	`a + b`
`-`	Soustraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Modulo (reste de division)	`a % b`

Exemple :

```
int a = 10;
int b = 3;
Console.WriteLine($"Addition : {a + b}");
Console.WriteLine($"Division : {a / b}");
Console.WriteLine($"Modulo : {a % b}");
```

Opérateurs de comparaison

Opérateur	Description	Exemple
`==`	Égalité	`a == b`
`!=`	Différent	`a != b`
`>`	Supérieur	`a > b`
`<`	Inférieur	`a < b`
`>=`	Supérieur ou égal	`a >= b`
`<=`	Inférieur ou égal	`a <= b`

Opérateurs logiques

Opérateur	Description	Exemple
`&&`	ET logique	`a > 0 && b > 0`
`||`	OU logique	`a > 0 || b > 0`

Opérateur	Description	Exemple
`!`	NON logique	`!(a > 0)`

---

6. Les Structures de Contrôle

6.1 Structures conditionnelles

Les **conditions** permettent d'exécuter du code en fonction de tests.

6.1.1 Condition if/else

```
int nombre = 5;

if (nombre > 0)
{
    Console.WriteLine("Le nombre est positif");
}
else if (nombre < 0)
{
    Console.WriteLine("Le nombre est négatif");
}
else
{
    Console.WriteLine("Le nombre est zéro");
}
```

6.1.2 Condition switch

```
int jour = 2;

switch (jour)
{
    case 1:
        Console.WriteLine("Lundi");
        break;
    case 2:
        Console.WriteLine("Mardi");
        break;
    default:
        Console.WriteLine("Jour inconnu");
        break;
}
```

6.2 Structures répétitives

6.2.1 Boucle for

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Tour : {i}");
}
```

6.2.2 Boucle while

```
int compteur = 1;
while (compteur <= 5)
{
    Console.WriteLine($"Compteur : {compteur}");
    compteur++;
}
```

6.2.3 Boucle do-while

```
int compteur = 1;
do
{
    Console.WriteLine($"Compteur : {compteur}");
    compteur++;
} while (compteur <= 5);
```

---

7. Exercices Pratiques

Exemple 1 : Calcul de la somme de deux nombres

Enoncé : Écrire un programme qui demande deux nombres à l'utilisateur et affiche leur somme.

Solution :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Entrez le premier nombre :");
        int nombre1 = int.Parse(Console.ReadLine());

        Console.WriteLine("Entrez le deuxième nombre :");
        int nombre2 = int.Parse(Console.ReadLine());
```

```
        int somme = nombre1 + nombre2;
        Console.WriteLine($"La somme est : {somme}");
    }
}
```

Exemple 2 : Vérification de la parité d'un nombre

Enoncé : Écrire un programme qui vérifie si un nombre est pair ou impair.

Solution :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Entrez un nombre :");
        int nombre = int.Parse(Console.ReadLine());

        if (nombre % 2 == 0)
        {
            Console.WriteLine($"{nombre} est pair");
        }
        else
        {
            Console.WriteLine($"{nombre} est impair");
        }
    }
}
```

Exercice 3 : Calcul du Factoriel d'un Nombre (Utilisation de Boucles et Conditions)

Énoncé :
Écrire un programme qui demande un nombre à l'utilisateur et affiche son **factoriel** (le produit des entiers de 1 à n).

Solution :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Entrez un nombre pour calculer son factoriel :");
        int nombre = int.Parse(Console.ReadLine());
```

```
        int factoriel = 1;

        for (int i = 1; i <= nombre; i++)
        {
            factoriel *= i;
        }

        Console.WriteLine($"Le factoriel de {nombre} est {factoriel}");
    }
}
```

Explication :

- Une boucle `for` multiplie chaque entier de 1 à `nombre`.
- La variable `factoriel` est initialisée à 1 pour éviter la multiplication par zéro.

Exercice 4 : Deviner un Nombre (Utilisation de Boucle While et Conditions)

Énoncé :
Créer un jeu où l'utilisateur doit deviner un nombre entre 1 et 100 choisi aléatoirement par l'ordinateur. Le programme doit lui indiquer si la valeur entrée est trop grande, trop petite, ou correcte.

Solution :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Random random = new Random();
        int nombreSecret = random.Next(1, 101);  // Génère un nombre entre 1 et
100
        int nombreDevine = 0;

        Console.WriteLine("Devinez le nombre secret entre 1 et 100 :");

        while (nombreDevine != nombreSecret)
        {
            nombreDevine = int.Parse(Console.ReadLine());

            if (nombreDevine < nombreSecret)
            {
                Console.WriteLine("C'est plus !");
            }
            else if (nombreDevine > nombreSecret)
            {
                Console.WriteLine("C'est moins !");
            }
```

```
            else
            {
                Console.WriteLine("Bravo ! Vous avez trouvé le nombre secret.");
            }
        }
    }
}
```

Explication :

- L'utilisation de `Random` permet de générer un nombre aléatoire.
- Une boucle `while` maintient le jeu jusqu'à ce que l'utilisateur devine correctement.

Exercice 5 : Gestion de Menu avec `switch`

Énoncé :
Créer un programme qui affiche un menu simple pour une calculatrice. L'utilisateur choisit une opération (Addition, Soustraction, Multiplication, Division), saisit deux nombres, et le programme affiche le résultat correspondant. Si une option invalide est choisie, afficher un message d'erreur.

Solution :

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Menu de la Calculatrice :");
        Console.WriteLine("1. Addition");
        Console.WriteLine("2. Soustraction");
        Console.WriteLine("3. Multiplication");
        Console.WriteLine("4. Division");
        Console.WriteLine("Entrez votre choix :");

        int choix = int.Parse(Console.ReadLine());

        Console.WriteLine("Entrez le premier nombre :");
        double nombre1 = double.Parse(Console.ReadLine());
        Console.WriteLine("Entrez le deuxième nombre :");
        double nombre2 = double.Parse(Console.ReadLine());

        switch (choix)
        {
            case 1:
                Console.WriteLine($"Résultat : {nombre1 + nombre2}");
                break;
            case 2:
                Console.WriteLine($"Résultat : {nombre1 - nombre2}");
                break;
```

```
            case 3:
                Console.WriteLine($"Résultat : {nombre1 * nombre2}");
                break;
            case 4:
                if (nombre2 != 0)
                    Console.WriteLine($"Résultat : {nombre1 / nombre2}");
                else
                    Console.WriteLine("Erreur : Division par zéro !");
                break;
            default:
                Console.WriteLine("Choix invalide !");
                break;
        }
    }
}
```

Explication :

- Le menu utilise une structure `switch` pour choisir une opération.
- Une vérification est faite pour éviter la division par zéro.

Exercices à faire

1. **Comparaison de deux nombres**
   Demandez deux nombres à l'utilisateur et affichez le plus grand.

2. **Calcul de la moyenne**
   Écrire un programme qui calcule la moyenne de trois nombres saisis par l'utilisateur.

3. **Multiplication avec une boucle for**
   Demandez un nombre à l'utilisateur et affichez sa table de multiplication jusqu'à 10.

4. **Compteur avec boucle while**
   Écrivez un programme qui affiche les nombres de 1 à 10 avec une boucle `while`.

Conclusion

Dans le prochain cours, nous aborderons les **structures de données proposées par C#**, notamment :

- Les **tableaux (`Array`)**
- Les **listes (`List`)**
- Les **dictionnaires (`Dictionary`)**
- Les **collections génériques**

Ces structures sont essentielles pour manipuler efficacement des données en mémoire.